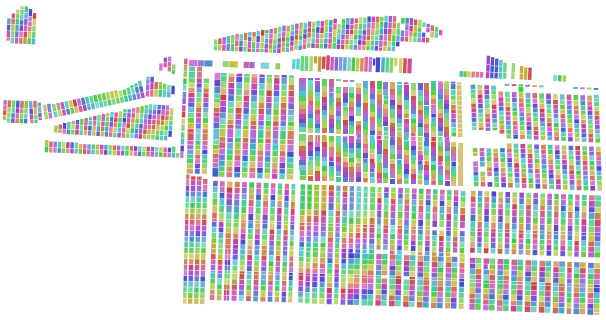
Cedar Grove H-Numbering, Explained

How the old lot numbers worked, why they were hard to use, and the arithmetic behind the H-number system.

The old numbers restart in every section. The H-numbers count one lot at a time along a single path. Here is how, and why it works.

Old numbers

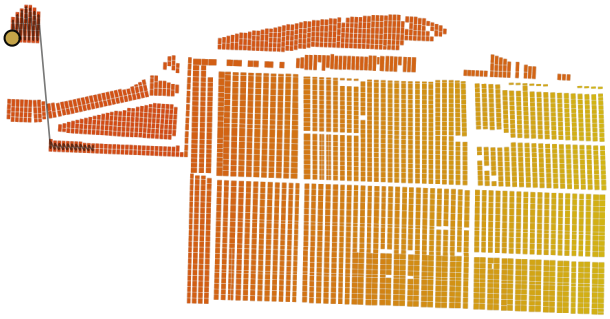
Each lot coloured by the number written on it. Every section starts again, so colours repeat and jump.



L-102 L-103 L-104 L-105 L-106 L-107

H-numbers

One path through every lot. Watch the number climb by one at each step.



H003185 of 3,185

Legacy label

19A

What is written on or beside a lot. Only unique inside its section.

Physical lot

one plot of ground

The outline drawn on the map. The map has 3,103 lot outlines.

H-number position

H001698

The lot's place along one fixed path, from H000001 to H003185.

Grave

H0016983

One of nine places inside a lot. The lot's H-number gets one more digit.

These counts differ on purpose. Several legacy references can name one lot (the two halves of a double-row lot), and some map outlines are not matched to a reference yet (see the unresolved list in the project folder).

AGREEMENT WITH THE DECODER

The formulas on this page reproduce the project decoder's H-number for all **3,197** real legacy labels (**3,199** entries when two internal bookkeeping keys are counted). This

> shows the formulas match the decoder. It does not show that the decoder matches the ground; section 09 separates those questions.

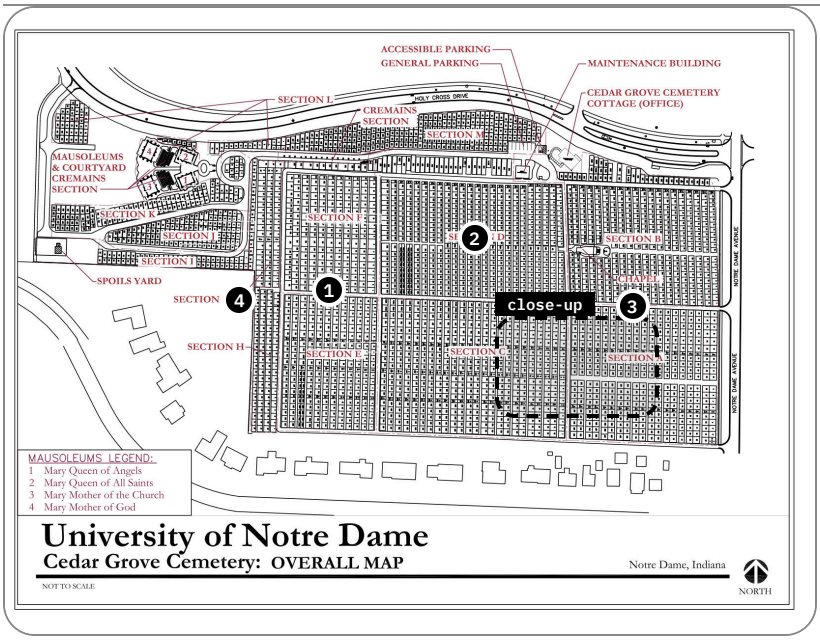
01

The raw maps

These are the maps the cemetery worked from, in the order a person would use them. Click any image to enlarge it.

1 Overall map **2** Section map **3** Close-ups **4** Working annotations

1 · Overall map

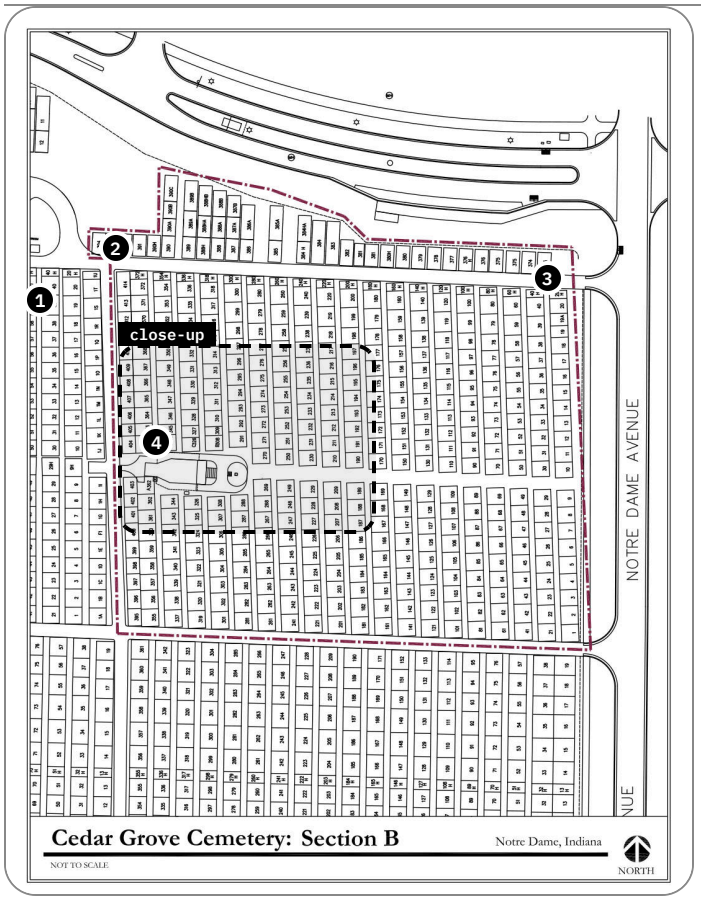


OVERALL MAP · ORIGINAL PDF

Every lot is drawn, and none carries a number.

1. No lot has a number on this map. A lot can only be found by its section.
2. Section titles are drawn on top of lots, so they hide the lots they name.
3. Section borders are not drawn. The line between B and A has to be inferred.
4. Narrow sections such as G and H are named by long leader lines.

2 · Section map

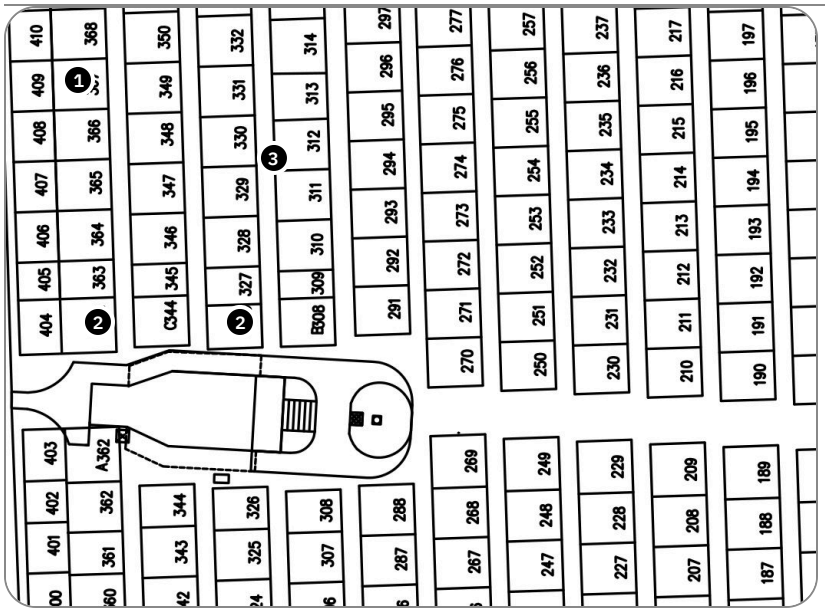


SECTION B · "ALL SECTIONS ON INDIVIDUAL PAGES"

Numbers appear here, at a few points tall.

1. Numbers are written sideways, rotated a quarter turn from the page.
2. The section boundary is a dash-dot line that cuts through some lots.
3. Lots near the top carry an extra "H" stacked under the number.
4. A few labels begin with another section's letter. See the close-up.

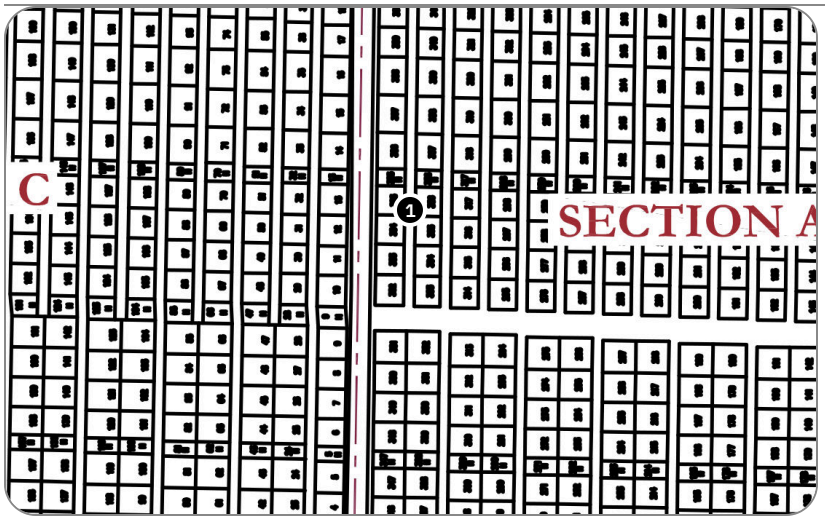
3 · Close-ups



SECTION B CLOSE-UP · 4×

Cut from the section map above (dashed rectangle).

1. Digits are rotated and tightly packed.
2. B362, C344, A362, C326 and B308 are lots in Section B whose labels start with another section's letter.
3. Neighbouring columns count up separately: 404, 405, 406 next to 363, 364, 365 next to 345, 346.

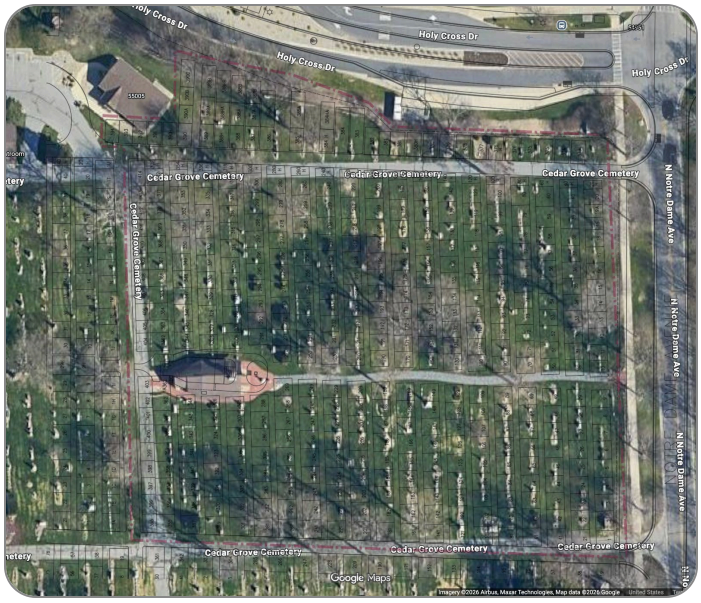


OVERALL MAP CLOSE-UP · 5×

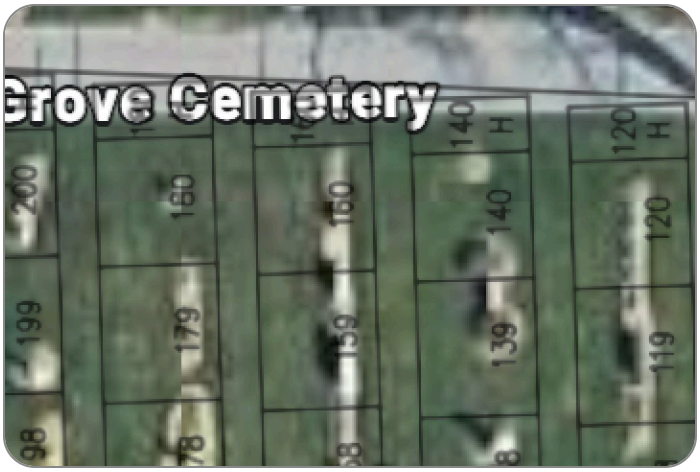
Cut from the overall map above (dashed rectangle).

1. Even enlarged, the boxes carry no numbers, so the overall map cannot be used to look up a lot.

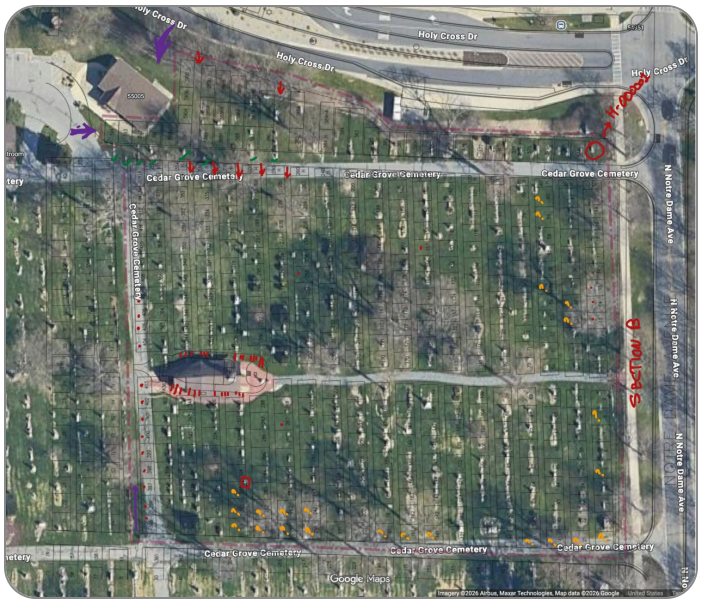
4 · Working annotations



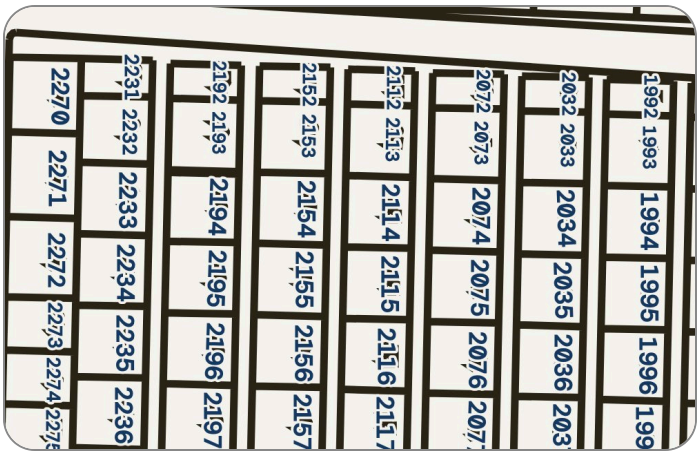
Aerial photo with the lot grid drawn on top (screenshot saved 2026-04-23). Numbers are a few pixels tall.



The same screenshot at native resolution, enlarged 3 times. Numbers sit over grass and headstones.



A hand-marked working copy of Section B (SECTION B.pdf): arrows, question marks, and a circled corner lot with a handwritten "H-00001".



An earlier digital version of this project: H-numbers drawn over the old number shapes, so the two overlap.

02

The old numbering

A legacy lot is named by a **section letter** and a **label.** The full address is both together. The label alone is not enough.

Anatomy of an old address

A	13	H
SECTION	LOT LABEL	SUFFIX

Section A. The section letter, A to M. Required: the same label exists in other sections. The full address of this lot is **A-13H**.

Real labels the decoder accepts

A-13H · B-19A · B-384HA · C-51 · D-1H · ! D-F1 · ! E-199BH · G-19 · K-57 · M-61 · D-384A

Click a label to see its address and H-number.

The first two rows of every section

Labels are shown in the order a person would walk them. Each row runs left to right. Click any label for its address and H-number.

19

Plain number

Just a number.

13H

H-lot

An extra lot squeezed in beside lot 13. This old "H" has nothing to do with the new H-numbers.

19A

Letter suffix

A or B marks one half of a double-row lot.

! D-F1

Unusual label

Starts with a letter or follows no pattern.

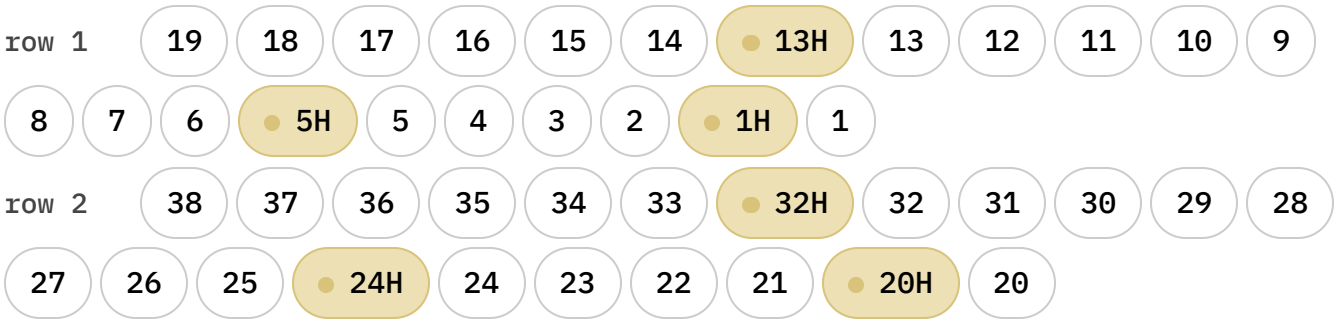
! gap

Gap

A place in the row with no label in the old records.

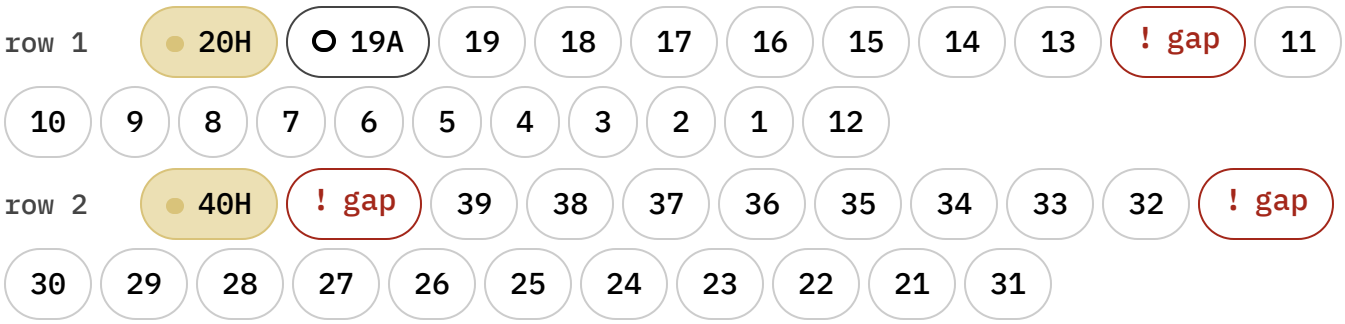
Section A

first two rows, in walking order



Section B

first two rows, in walking order



Section C

first two rows, in walking order



Section D

first two rows, in walking order



! 1L ! 1K ! 1J ! 1I ● 1H 20 19 18 17 16 15 14
13 12 11 10 9 8 7 6 5 4 3 2 1 ○ 1A ○ 1B
○ 1C ! 1D ! 1E ! F1 ! 1G ! 1H_2 ! 1I_2
row 2 ● 20H 40 39 38 37 36 35 34 33 32 31 30
● 29H 29 28 27 26 25 24 23 22 21

Section E

first two rows, in walking order

row 1 19 18 17 16 15 14 ● 13H 13 12 11 10 ● 9H
9 8 7 6 ● 5H 5 4 3 2 1
row 2 38 37 36 35 34 33 ● 32H 32 31 30 29
● 28H 28 27 26 25 ● 24H 24 23 22 21 20

Section F

first two rows, in walking order

row 1 18 17 16 15 14 13 12 11 10 9 8 7 6
5 4 3 2 1
row 2 36 35 34 33 32 31 30 29 28 27 26 25
24 23 22 21 20 19

Section G

first two rows, in walking order

row 1 58 19 19 18 17 16 15 14 13 12 11 10 9

8H 8 7 6 5 4 3 2 1

row 2 59 38H 38 37 36 35 34 33 32 31 30 29

28 27H 27 26 25 24 23 22 21 20

Section H

first two rows, in walking order

row 1 23 22 21 20 19 18 17 16 15 14 13 12

11 10 9 8 7 6 5 4 3 2 1

row 2 46 45 44 43 42 41 40 39 38 37 36 35

34 33 32 31 30 29 28 27 26 25 24

Section I

first two rows, in walking order

row 1 1 2

row 2 3 4

Section J

first two rows, in walking order

row 1 1 2 3 4 5

row 2 6 7 8 9 10 11 12

Section K

first two rows, in walking order

row 1 1 2 3

row 2 6 7 8 9

Section L

first two rows, in walking order

row 1 69 70 71 72 73

row 2 74 75 76 77 78 79

Section M

first two rows, in walking order

row 1 11 12

row 2 14 15 16

Click a label above.

03 Why it confuses

Three different kinds of problem are mixed together here. They need different fixes, so this section keeps them apart.

A

Reading problems

The number is hard to see or place. Fixed by redrawing the map.

B

Section rules

Each section counts in its own way. Fixed by numbering everything the same way.

C

Data inconsistencies

Typos, duplicates and conflicts. Fixed only by a person deciding what is right.

A Reading problems

- The overall map has no lot numbers at all (section 01, image 1).
- Where numbers exist they are a few points tall and rotated (section 01, images 2 and 3).
- Section titles and notes cover lots, and section borders are not drawn.

B Section-specific rules

3,197	legacy references the decoder accepts
768	distinct label strings among them
474	labels used by two or more sections
91%	of references share their label string with another section
12	sections that each have a lot labelled ”19”
36	positions in the first sweep with no label at all

A repeated label is not a duplicate address. "B-19" and "C-19" are different complete addresses. The cost is that a label alone never says where a lot is. Every lookup needs the section too, and each section has its own counting rule.

Try it: one label, many places

LEGACY LABEL

19

12 lots labelled "19", one per section:

- B-19 → H000003
- A-19 → H000023
- D-19 → H000852
- C-19 → H000880
- E-19 → H001890
- F-19 → H001929
- G-19 → H002415
- H-19 → H002440
- M-19 → H002593
- K-19 → H002852
- J-19 → H002969
- I-19 → H003101

Each is a different complete address. To use the label you must also know the section and that section's rule.

How each section counts

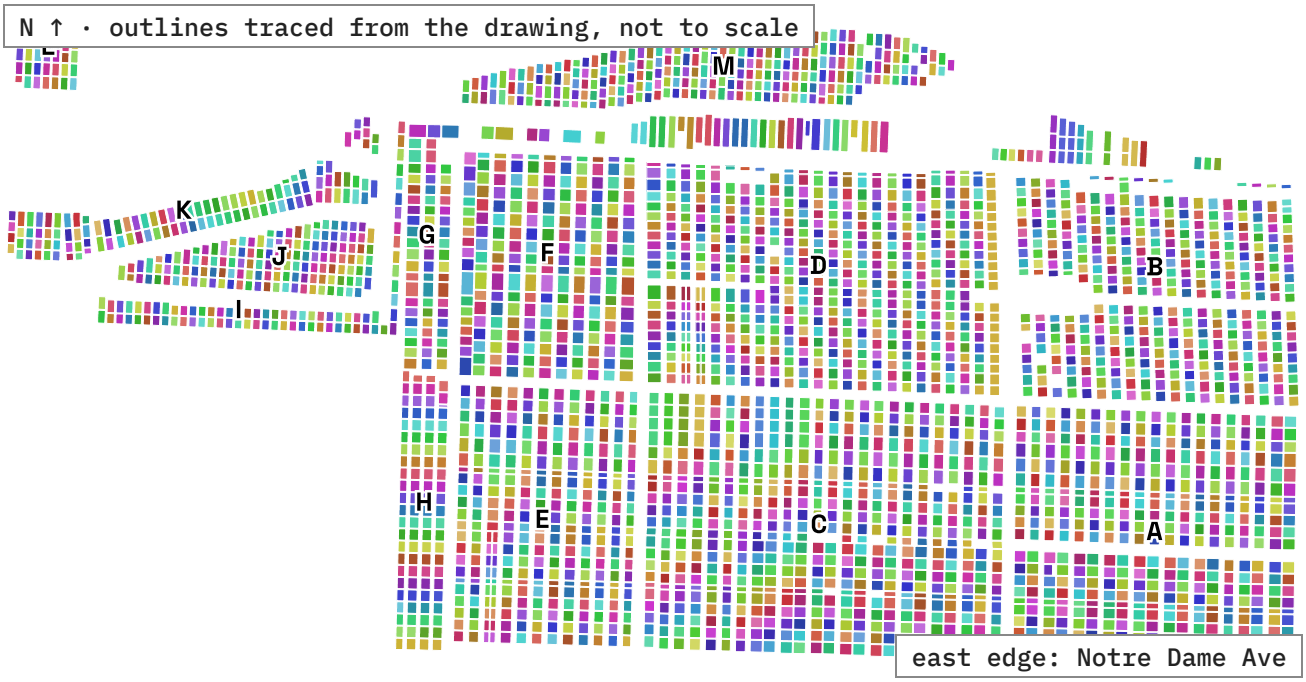
- **Arithmetic** : one formula covers every lot
- **Row constants** : a formula plus a small table of numbers for each row
- **Explicit list** : no formula, the order is stored lot by lot
- **Errata** : a known typo or duplicate

SECTION	HOW ITS LABELS ARE BUILT	SPECIAL CASES	DECODER COVERAGE
A	19 per row, counted right to left	H-lots slipped in before places 13, 5 and 1 of every row	418 arithmetic
B	20 per row in the body, last column steps by 19	Columns 2 and 3 both claim 19; "19A" is an extra; a 41-lot margin is numbered 373 to 394 with letters	374 arithmetic, 41 list
C	Row tops go 19, 38 … 247, then 16 at a time, then 19	Twin pairs (51 and its H-lot) and A/B/C trios on eight rows	518 row constants, 1 errata
D	20 per row for rows 2 to 15, then blocks that run both ways	H-lots at 20(r−1) and 20r−11; row 1 uses letters; a 29-lot margin numbered 542 to 563	336 arithmetic, 244 row constants
E	19 per row with twins at places 13, 9 and 5	Rows 9 to 12 change shape; lots 152, 169 and 190 to 199 carry A/B suffixes	176 arithmetic, 110 row constants
F	18 per row	Eleven rows; a margin of 12 lots numbered 199 to 210	210 arithmetic
G	19 per row for lots 1 to 57	58 to 60 and 61 to 75 are added on; the label 19 appears twice in row 1	79 row constants, 1 errata
H	23 per row	Four rows only	92 arithmetic
I	1 to 64 straight	Just a counter	64 arithmetic
J	1 to 132 straight	Just a counter	132 arithmetic
K	115 lots in an order no formula gives	1, 2, 3, 6, 7, 8, 9, 13 … stored as a list	115 list
L	Starts at 69 and ends at 107	Numbers 1 to 68 do not exist here	39 arithmetic
M	82 to 279 straight	The first 51 lots are in an irregular order and stored as a list	198 arithmetic, 51 list

The whole cemetery, colored

Each lot is colored by its old label number, its old section, or its H-number.

Colored by the number written on the lot. Neighbouring lots get unrelated colors because every section restarts and skips.



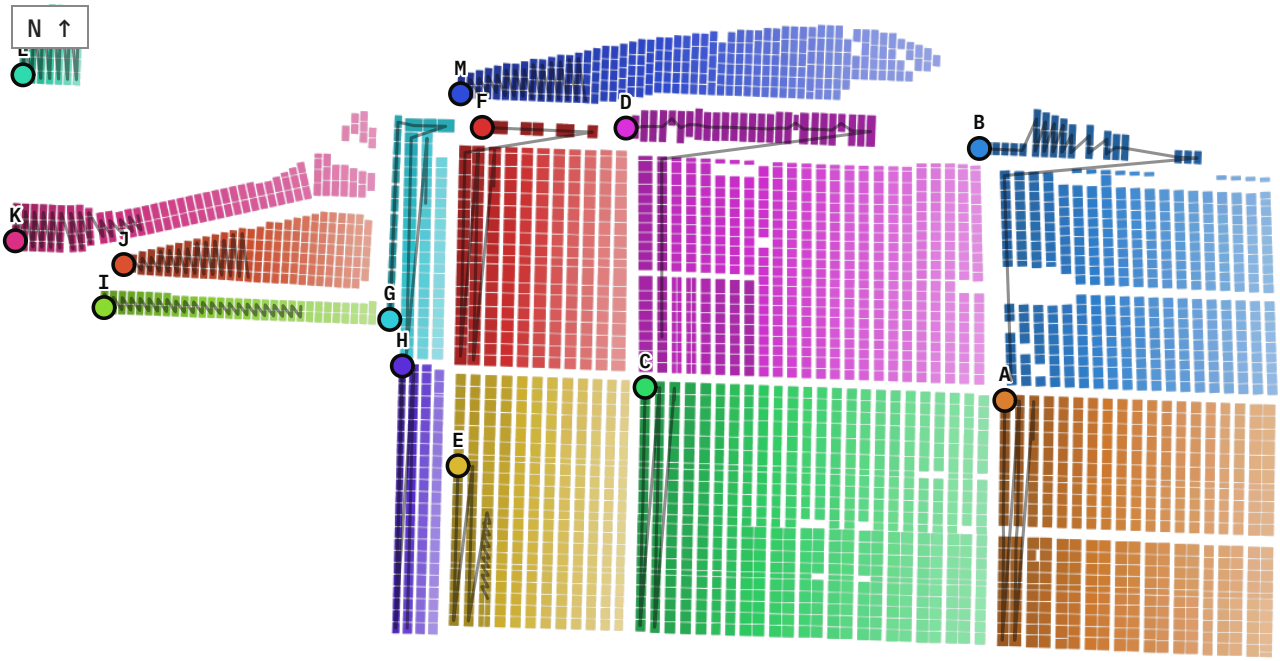
Each color is a label number. Colors repeat because label numbers repeat in every section.

Geometry: lot outlines traced from the cemetery drawing (about 3,100 outlines). Positions follow the drawing, which is marked "not to scale".

Walk both numbering systems

The same cemetery, counted two ways. Press play and compare.

The old way. A is numbered on its own, B on its own, and so on. Every section starts again at its own first lot, so the walkers run side by side and never agree.



All 13 sections at once. Each section counts from its own first label, so they walk in different directions at the same time. Dark means near the end.





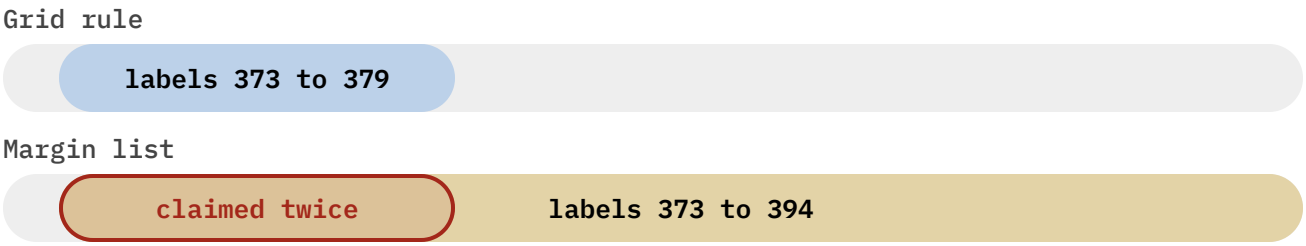
C Data inconsistencies

Seven places where the old records contradict themselves. Each card shows the problem, then the one thing we need to know.

7 problems found **5** need a decision **2** handled

Needs a decision SECTION B

Two lists use the same labels



7 labels (373 to 379) appear in both lists

Section B counts its grid and its margin with overlapping numbers. B-373 means a lot in the margin, not the grid.

We need to know: Which lot does each of these 7 labels belong to?

Needs a decision SECTION B, A

Positions with no old label



- each red mark is a position on the path (H000001 to H003185) with no old label, 46 in all

46 places on the path have no old label. They can only be found by H-number or by counting on the ground.

We need to know: Does a real lot stand at each of these positions?

Needs a decision SECTION D, G

One label, two lots

Section G, row 1

17	18	19	19	20	19H?

Section D table

273	274	274	275

- two lots answer to one label, so the software can only reach one of them

Two entries in the old tables share a label, so one lot can never be looked up.

We need to know: Which lot did each entry mean?

Needs a decision SECTION C

A typo in a row table

Pattern says

143	142	142H	141

Table says

143	142	124H	141

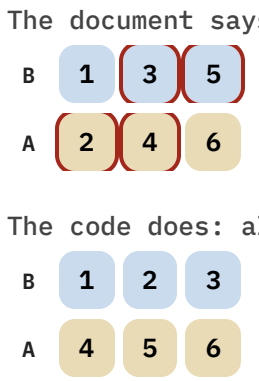
- two digits swapped in row 8 of Section C

Row 8 lists 124H where the pattern needs 142H. A search for 142H finds nothing.

We need to know: Is the lot on the map really 142H?

Needs a decision SECTION B, A

The paper and the code disagree



- the numbers in red land on different lots depending on which order is right

The reference PDF alternates B and A column by column. The code does a whole level of B first. Every H-number after the first lot depends on this.

We need to know: Which order is the official one?

Handled SECTION C

The idealised grid is wrong



- the sheet is 199 lots short, because it assumes every row has 22 lots

The reference sheet assumes 22 lots in every row. Real rows vary. The decoder already uses the real rows.

Still to do: Retire or correct the old sheet.

Handled SECTION H, M

Where H ends and M begins



What older notes said

`M starts H002586`

- one position off, so the last lot of H lands in the wrong pass

Section H ends at H002586 and M starts at H002587. Earlier summaries quoted M at H002586.

Still to do: Check any document that quotes the old boundary.

04

The H-number idea

Number lots by **where they fall along one fixed path** through the cemetery. The first lot on the path is H000001, the next is H000002, and so on to H003185.

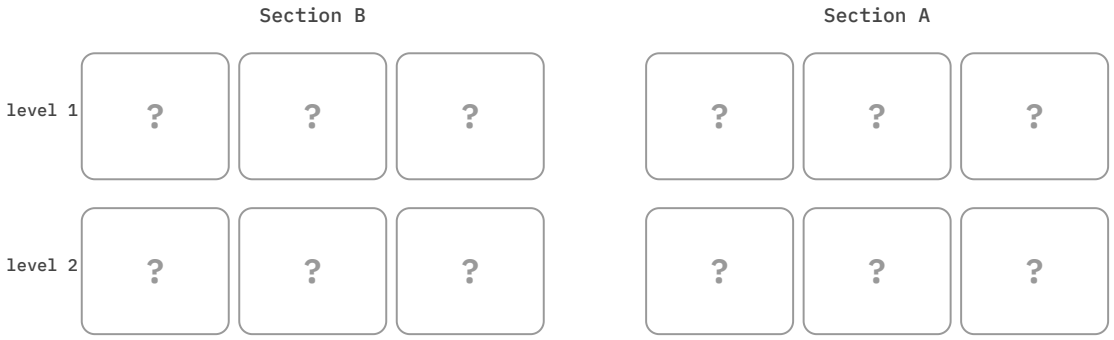
See the path before the equation

ILLUSTRATION · 12 LOTS

QUESTION

How can twelve lots in two sections get one running number?

DIAGRAM



WORKING

Press Play or Step. Two sections (B, A) of 3 columns, 2 levels: 12 lots.

> Takeaway: a lot's H-number is how many lots the path has already visited, plus one.

The three rules

1. The path covers the cemetery in six stretches, called **passes**, one after another.
2. Inside a pass it goes **level** by level, and along each level it visits the lots in a fixed order.
3. A lot's H-number is the number of lots already visited, plus one.

$$H_{\text{lot}} = O + \sum_{j<r} W_j + k$$

O lots in earlier passes. $\mathbf{W_j}$ lots on level j of this pass. **r** the level of this lot. **k** its place on that level.

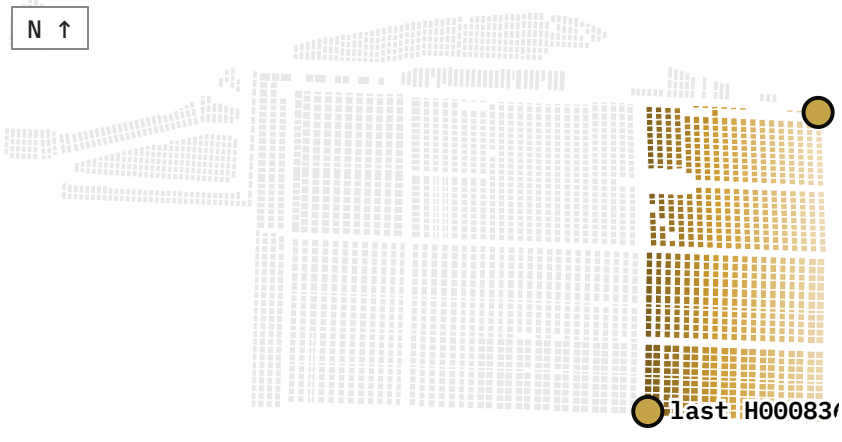
Words used on this page

- **Pass**: one of six stretches of the path.
- **Level** (also called row): one step of a pass. A level lists lots from one or two sections.
- **Column** and **place**: a lot's position on its level.

Level, row and column are **decoder indexing**. They count positions in the decoder's tables. They are not always the compass rows and columns on the ground, and some sections are drawn rotated.

The six passes on the map

Select a pass to see which part of the cemetery it covers. The gold runs from light (the first lot) to dark (the last lot). Press Simulate to watch the path fill in, pass by pass.



Pass 1 · sections B, A

```
H000001 to H000836
836 positions
764 have an outline on the map
```

Level by level: one level of B, then the same level of A. 19 levels of 44 lots.

first lot to last lot. Lots outside the pass are gray.

PASS	SECTIONS	H-NUMBERS	POSITIONS	SHARE OF 3,185
1	B, A	H000001 – H000836	836	
2	D, C	H000837 – H001871	1,035	
3	F, E	H001872 – H002331	460	
4	B, D, F margins; G, H	H002332 – H002586	255	
5	M	H002587 – H002835	249	
6	K, J, I, L	H002836 – H003185	350	

Where two sections share a pass, their levels are woven: level 1 of the first section, level 1 of the second, then level 2 of each. That is why one section's H-numbers are not one block. Pass 4 holds three margin lists and the G/H rows; the margins are numbered before G and H.

Reading a full address

TRY AN ADDRESS

H001698A3

H	001698	A	3
PREFIX	LOT (H-NUMBER)	DOUBLE-ROW HALF	GRAVE 1 TO 9

Lot H001698 is position 1698 of 3,185 on the path. Old references: D-376, D-376A, D-376B.

The numeric form 10 × lot + g would give 16983, which has lost the letter A. Keep the letter in the address.

$$H_{\text{grave}} = 10 \times H_{\text{lot}} + g$$

The numeric form holds only for lots without an A or B. A double-row lot keeps its letter between the lot number and the grave digit (H001698A3), and the numeric form cannot carry that letter. Lot H000001 has graves H0000011 to H0000019.

05

The math, step by step

Six demonstrations, each laid out the same way: question, then diagram beside its controls, then the working, then the takeaway. Each starts on the ACTUAL SYSTEM settings. Move a control and it becomes an EXPERIMENT. It does not change the real numbers. Press Reset this example to return.

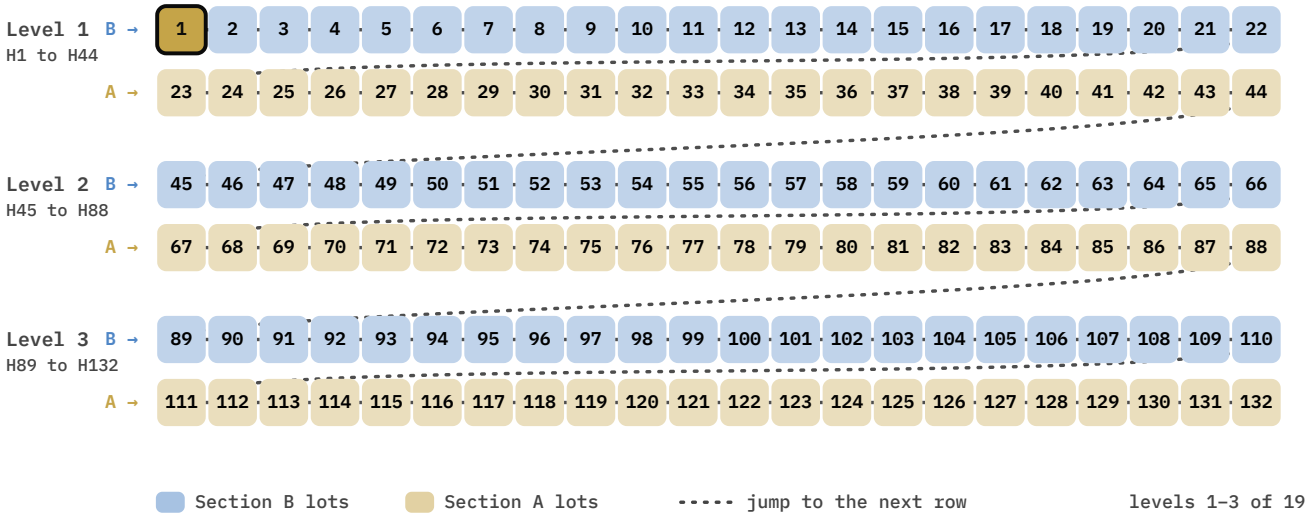
Step 1 · The two-section sweep (B and A)

ACTUAL SYSTEM

QUESTION

Sections B and A are walked together. Which lot comes first, and what is the H-number of any lot you pick?

DIAGRAM



How to read it. Each box is one lot and the number inside is its H-number. The path runs along the B row, jumps to the A row (dotted line), runs along it, then drops to the next level. Click any box to pick it.

WORKING

$$\text{SWEEP} = N_B + N_A = 44$$

$$\text{OFFSET} = \text{SWEEP} \times R = 836$$

$$H(B, r, c) = \text{SWEEP} \times (r - 1) + c$$

$$H(A, r, c) = \text{SWEEP} \times (r - 1) + N_B + c$$

H(B, level 1, column 1) = 44 × 0 + 1 = 1 → H000001
Visited as lot 1 of 836 in this pass. On level 1 the sweep visits B columns 1 to 22 (H 1 to 22), then A columns 1 to 22 (H 23 to 44).

> Takeaway: the reference PDF and decoder docstring describe the order as alternating columns, but the code and every verified number use a whole level of B first, then the same level of A. Switch the order to see how that one choice moves every H-number after the first lot.

Step 2 · H-lots push labels along

ACTUAL SYSTEM

QUESTION

In Section A, how do three H-lots per row change which column each label lands in?

DIAGRAM

Row 1, without H-lots: label at column

19@c1 · 18@c2 · 17@c3 · 16@c4 · 15@c5 · 14@c6 · 13@c7 · 12@c8 · 11@c9 · 10@c10 · 9@c11 · 8@c12 · 7@c13 · 6@c14 · 5@c15 · 4@c16 · 3@c17 · 2@c18 · 1@c19

Row 1, with H-lots inserted (gold dot), in H order. A ring marks a label pushed one column over.

19@c1 → 23 · 18@c2 → 24 · 17@c3 → 25 · 16@c4 → 26 · 15@c5 → 27 · 14@c6 → 28 · ● 13H@c7 → 29 · ○ 13@c8 → 30 · ○ 12@c9 → 31 · ○ 11@c10 → 32 · ○ 10@c11 → 33 · ○ 9@c12 → 34 · ○ 8@c13 → 35 · ○ 7@c14 → 36 · ○ 6@c15 → 37 · ● 5H@c16 → 38 · ○ 5@c17 → 39 · ○ 4@c18 → 40 · ○ 3@c19 → 41 · ○ 2@c20 → 42 · ● 1H@c21 → 43 · ○ 1@c22 → 44

Row 2, without H-lots: label at column

38@c1 · 37@c2 · 36@c3 · 35@c4 · 34@c5 · 33@c6 · 32@c7 · 31@c8 · 30@c9 · 29@c10 · 28@c11 · 27@c12 · 26@c13 · 25@c14 · 24@c15 · 23@c16 · 22@c17 · 21@c18 · 20@c19

Row 2, with H-lots inserted (gold dot), in H order. A ring marks a label pushed one column over.

38@c1 → 67 · 37@c2 → 68 · 36@c3 → 69 · 35@c4 → 70 · 34@c5 → 71 · 33@c6 → 72 · ● 32H@c7 → 73 · ○ 32@c8 → 74 · ○ 31@c9 → 75 · ○ 30@c10 → 76 · ○ 29@c11 → 77 · ○ 28@c12 → 78 · ○ 27@c13 → 79 · ○ 26@c14 → 80 · ○ 25@c15 → 81 · ● 24H@c16 → 82 · ○ 24@c17 → 83

○ 23@c18 → 84 ○ 22@c19 → 85 ○ 21@c20 → 86 ● 20H@c21 → 87 ○ 20@c22 → 88

No two lots share an H-number.

WORKING

```
row r = ⌈ n / step ⌉ place q = n − step × (r − 1)
column c = (N − 2 − q) + [q ≤ t₁] + [q ≤ t₂] + … an H-lot sits one column earlier
H(A, n) = SWEEP × (r − 1) + N_B + c
```

> Takeaway: every H-lot slipped in before a place shifts all later labels one column. That shift is why an old label cannot be turned into H without knowing where the H-lots sit.

Section B, where the column rules disagree

B's reference formulas give column 2 the label 19 + 20(r − 1), and column 3 (the first body column) also gets 19 + 20(r − 1). The body steps by 20 per row and the last column by 19. The decoder lets the later rule overwrite the earlier one.

BODY STEP: 20 LAST-COLUMN STEP: 19 LAST-COLUMN FIRST LABEL: 12

```
row 1 body labels: 1 … 19
last column: 12, 31, … , 354
18 last-column labels collide with a body label (12, 31, 50, 69, 88, 107 …)
Real values are 20, 19 and 12; collisions there are resolved by "later rule wins".
```

Step 3 · Rows of different widths (D and C)

ACTUAL SYSTEM

QUESTION

When rows differ in length, what happens to every lot after a row that gains or loses one lot?

DIAGRAM

ROW	W_D	W_C	D STARTS AT	C STARTS AT
1	43	21	837	880
2	22	21	901	923
3	22	22	944	966
4	22	21	988	1010
5	22	21	1031	1053
6	22	22	1074	1096
7	22	22	1118	1140
8	22	22	1162	1184
9	22	20	1206	1228
10	22	22	1248	1270
11	22	22	1292	1314
12	22	20	1336	1358

Shaded cells show rows whose starting H-number shifts compared with the real system after your edit.

WORKING

$$H(D, \text{row } r, \text{place } k) = 836 + \sum_{j<r} (W_D(j) + W_C(j)) + k$$

$$H(C, \text{row } r, \text{place } k) = 836 + \sum_{j<r} (W_D(j) + W_C(j)) + W_D(r) + k$$

H(D, row 5, place 3) = 836 + 194 + 3 = 1033 → H001033

Pass 2 ends at 1871 (matches the real 1871)

Takeaway: a row's offset is the running total of every earlier row, so one extra lot in row 5 moves every later lot by one. The older sheet assumed 22 lots in every row and ended C-417 at H001672. The real rows are uneven, so the decoder ends it at H001871.

Step 4 · Twins and trios inside a row (C)

ACTUAL SYSTEM

QUESTION

How do an H-lot beside a plain lot, or an A, B, C trio at a row's start, change the places of the lots after it?

DIAGRAM

Row 3 of C, top lot 57, in H order. Gold dot = H-lot, ring = trio lot.

57 → 966	56 → 967	55 → 968	54 → 969	53 → 970	52 → 971
● 51H → 972	51 → 973	50 → 974	49 → 975	48 → 976	● 47H → 977
47 → 978	46 → 979	45 → 980	44 → 981	● 43H → 982	43 → 983
42 → 984	41 → 985	40 → 986	39 → 987		

22 of these labels exist in the decoder; all agree with it

WORKING

```
k(plain lot n) = (top − n) + 1 + 3·[trio row] + #{ twins t : t > n, or t = n with its H-lot first }
k(H-lot t) = k(plain t) − 1 (H-lot first) or k(plain t) + 1 (H-lot after)
```

> Takeaway: each twin or trio lot pushes everything after it one place along, in the same way H-lots did in step 2.

Step 5 · Straight-line sections (M, J, I, L, F)

ACTUAL SYSTEM

QUESTION

Where a section was numbered straight through, what is H?

DIAGRAM

```
H(J, n) = 2950 + n (n = 1 … 132)
H(I, n) = 3082 + n (n = 1 … 64)
H(L, n) = 3146 + (n − 68) (n = 69 … 107)
H(M, n) = n + 2556 (n = 82 … 279; the first 51 lots of M come from a list)
H(F, n) = n + 2203 for the margin lots 199 … 210
```

WORKING

```
H(J, 57) = 3007 → H003007
rule: 2950 + n
derived by arithmetic (one formula)
The decoder returns the same value.
```

> Takeaway: for these sections H is the lot number plus a constant. The constants are where the section's pass starts.

Step 6 · Graves

ACTUAL SYSTEM

QUESTION

How does a lot's H-number become a grave address?

DIAGRAM

H	001698	A	3
PREFIX	LOT (H-NUMBER)	DOUBLE-ROW HALF	GRAVE

WORKING

```
address: H001698A3
numeric form 10 × 1698 + 3 = 16983 drops the letter A; keep it in the address.
old references to this lot: D-376, D-376A, D-376B
```

> Takeaway: the address is lot, then optional A or B, then grave digit. The numeric shortcut 10 × lot + g drops the letter, so use it only when there is no suffix.

06

The code

The real functions that turn an old address into an H-number. Tap a section to read its code.

A
H_A_grid
19 per row, H-lots

B
H_B_grid
20 per row, margin list

C
H_C
row tops, twins, trios

D
H_D
blocks and margin

E
H_E
twins, lettered lots

F
H_F
18 per row

G
H_G
19 per row

H
H_H
23 per row

I
H_I
a counter

J
H_J
a counter

K
H_K
a stored list

L
H_L
69 to 107

M
H_M
list, then a counter

build_B_decoder s4/s4decoder1.py · 13 lines · existing decoder

Builds the table that turns a Section B label into its column and level.

Implements: Step 1 · the B and A sweep

```
def build_B_decoder():
    d = {}
    for r in range(1, ROWS+1):
        d[f"{20*r}H"] = (1, r)
    d["19A"] = (2, 1)
    for r in range(2, ROWS+1):
        d[str(19 + 20*(r-1))] = (2, r)
    for r in range(1, ROWS+1):
        for c in range(3, 22):
            d[str((r-1)*20 + (22-c))] = (c, r)
    for r in range(1, ROWS+1):
        d[str(12 + 19*(r-1))] = (22, r)
    return d
```

build_A_decoder s4/s4decoder1.py · 13 lines · existing decoder

Builds the same table for Section A, including where the H-lots fall.

Implements: Step 2 · H-lots push labels along

```
def build_A_decoder():
    d = {}
    for r in range(1, ROWS+1):
        b = 19*(r-1)
        d[str(19+b)]=(1,r);  d[str(18+b)]=(2,r);  d[str(17+b)]=(3,r)
        d[str(16+b)]=(4,r);  d[str(15+b)]=(5,r);  d[str(14+b)]=(6,r)
        d[f"{13+b}H"]=(7,r); d[str(13+b)]=(8,r);  d[str(12+b)]=(9,r)
        d[str(11+b)]=(10,r); d[str(10+b)]=(11,r)
        d[str(9+b)] =(12,r); d[str(8+b)] =(13,r); d[str(7+b)] =(14,r)
        d[str(6+b)] =(15,r); d[f"{5+b}H"]=(16,r); d[str(5+b)] =(17,r)
        d[str(4+b)] =(18,r); d[str(3+b)] =(19,r); d[str(2+b)] =(20,r)
        d[f"{1+b}H"]=(21,r); d[str(1+b)] =(22,r)
    return d
```

`_build_dc_tables` s4/s4decoder1.py · 15 lines · existing decoder

Builds the running-total tables for sections D and C, so each row knows how many lots come before it.

Implements: Step 3 · rows of different widths

```
def _build_dc_tables():
    D_LOT_H, C_LOT_H, DC_H_LOT = {}, {}, {}
    h = OFFSET_DC
    for r in range(1, max(max(_D_ROWS), max(_C_ROWS)) + 1):
        for lot in _D_ROWS.get(r, []):
            h += 1
            if lot not in D_LOT_H:
                D_LOT_H[lot] = h
            DC_H_LOT[h] = ('D', lot)
        for lot in _C_ROWS.get(r, []):
            h += 1
            if lot not in C_LOT_H:
                C_LOT_H[lot] = h
            DC_H_LOT[h] = ('C', lot)
    return D_LOT_H, C_LOT_H, DC_H_LOT, h
```

`h_lot` s4/s4decoder1.py · 96 lines · existing decoder

Takes a section and a label and returns the lot's H-number. This is the function the one-line formula reproduces.

Implements: Section 07 · the giant formula

```
def h_lot(section, lot_label):
    """
    Return (h_number, 'H000837') for a section + lot label.

    Section D and C are table-driven from the exact map sweep order.
    Section E double-row lots return H{n}A or H{n}B suffix.
    All other sections use formula-based decoding.
    """
    sec = section.upper().strip()
    lot = str(lot_label).strip()

    # ── Pass 2: D and C (table-driven, Section 1 + Section 2) ──────
    if sec == 'D':
        # Double-row pair: '384A'/'376B' → same H as base lot, + suffix
        if len(lot) > 1 and lot[-1] in ('A','B') and lot[:-1] in _D_DR_PAIRS:
```

```
            base, suffix = lot[:-1], lot[-1]
            h = _D_LOT_H.get(base)
            if h is None:
                return None, f"Base lot '{base}' not found in Section D."
            return h, f"H{h:06d}{suffix}"
        # Section 1 main table first, then Section 2 margin table
        h = _D_LOT_H.get(lot) or _D2_LOT_H.get(lot)
        if h is None:
            return None, f"Lot '{lot}' not found in Section D."
        return h, f"H{h:06d}"

    if sec == 'C':
        h = _C_LOT_H.get(lot)
        if h is None:
            return None, f"Lot '{lot}' not found in Section C."
        return h, f"H{h:06d}"

    # ── Section 4: K  J  I  L ──────────────────────────────────────
    _S4_MAP = {'K':_K_LOT_H,'J':_J_LOT_H,'I':_I_LOT_H,'L':_L_LOT_H}
    if sec in _S4_MAP:
        h = _S4_MAP[sec].get(lot)
        if h is None:
            return None, f"Lot '{lot}' not found in Section {sec}."
        return h, f"H{h:06d}"

    # ── Section 3: M ───────────────────────────────────────────────
    if sec == 'M':
        h = _M_LOT_H.get(lot)
        if h is None:
            return None, f"Lot '{lot}' not found in Section M."
        return h, f"H{h:06d}"

    # ── Section 2: B margins + G/H ─────────────────────────────────
    # B margin lots (373-394) and G/H sections are checked here.
    # D margin lots are handled in the D block below.
    # F margin lots are handled in the F block below.
    if sec == 'B':
        # Check Section 2 margin lots first (373-394 etc.)
        h2 = _B2_LOT_H.get(lot)
        if h2 is not None:
            return h2, f"H{h2:06d}"
        # Fall through to formula-based Section 1 B lookup below
    elif sec in ('G', 'H'):
        _gh = _G_LOT_H if sec == 'G' else _H_LOT_H
        h = _gh.get(lot)
        if h is not None:
            return h, f"H{h:06d}"
        return None, f"Lot '{lot}' not found in Section {sec}."

    # ── Pass 3: F (table-driven, Section 1 + Section 2 margins) ──
    if sec == 'F':
        # Check Section 2 margins (199-210) first, then Section 1 table
        h = _F2_LOT_H.get(lot) or _F_LOT_H.get(lot)
        if h is None:
            return None, f"Lot '{lot}' not found in Section F."
        return h, f"H{h:06d}"

    # ── Pass 3: E (table-driven, A/B suffix for double-row pairs) ──
    if sec == 'E':
        if lot in _E_DR_SUFFIX_MAP:
            base = _E_DR_SUFFIX_MAP[lot]; suffix = lot[-1]
            h = _E_LOT_H.get(base)
```

```
            if h is None:
                return None, f"Double-row base '{base}' not found in Section E."
            return h, f"H{h:06d}{suffix}"
        h = _E_LOT_H.get(lot)
        if h is None:
            return None, f"Lot '{lot}' not found in Section E."
        return h, f"H{h:06d}"

    # ── Pass 1: B, A (formula-based) ──────────────────────────
    dec = _DECODERS.get(sec)
    if dec is None:
        return None, f"Unknown section '{section}'. Use A, B, C, D, E, or F."
    if lot not in dec:
        return None, f"Lot '{lot}' not found in Section {sec}."
    c, r = dec[lot]
    if sec == 'B':   h = SWEEP*(r-1) + c
    elif sec == 'A': h = SWEEP*(r-1) + N_B + c
    else: return None, f"Section '{sec}' has no formula."
    return h, f"H{h:06d}"
```

h_grave

`s4/s4decoder1.py · 21 lines · existing decoder`

Adds the grave digit (and the A or B letter) to a lot's H-number.

Implements: Step 6 · graves

```
def h_grave(section, lot_label, grave_number):
    """
    Return grave H number. For double-row lots, the suffix sits BETWEEN
    the lot number and the grave digit.
    e.g. h_grave('D', '376H', 3) → (1634, 'H001634A3')
         h_grave('D', '376B', 3) → (1634, 'H001634B3')
         h_grave('B', '20H',  1) → (1,    'H0000011')
    """
    h, hfmt = h_lot(section, lot_label)
    if h is None: return None, hfmt
    g = int(grave_number)
    if not (1 <= g <= 9): return None, f"Grave {g} out of range."
    # Detect A/B suffix from lot H string (e.g. 'H001634A')
    suffix = ''
    if hfmt and hfmt[0] == 'H' and hfmt[-1] in ('A', 'B'):
        suffix = hfmt[-1]
    hg = h * 10 + g
    if suffix:
        # Format: H{lot_base:06d}{suffix}{grave}  e.g. H001634A3
        return hg, f"H{h:06d}{suffix}{g}"
    return hg, f"H{hg:07d}"
```

decode_h

`s4/s4decoder1.py · 46 lines · existing decoder`

Goes the other way: an H-number back to its old section and label.

Implements: Section 07 · reverse lookup

```
def decode_h(h_number):
    """Reverse lookup: H number → {section, lot_label, h_number, h_formatted}"""
    n = int(h_number)
    max_h = _S4_MAX_H
    if n < 1 or n > max_h:
        return None

    if n <= OFFSET:                          # Pass 1 — B / A
        level = (n-1)//SWEEP; pos=(n-1)%SWEEP+1; r=level+1
```

```
        if pos<=N_B: c,sec,dec=pos,'B',B_DECODER
        else: c,sec,dec=pos-N_B,'A',A_DECODER
        tgt=(c,r)
        lot=next((k for k,v in dec.items() if v==tgt),None)
        return {"section":sec,"lot_label":lot,"c":c,"r":r,
                "h_number":n,"h_formatted":f"H{n:06d}"}

    elif n <= _DC_MAX_H:                    # Pass 2 — D / C (table-driven)
        entry=_DC_H_LOT.get(n)
        if entry is None: return None
        sec,lot=entry
        return {"section":sec,"lot_label":lot,
                "h_number":n,"h_formatted":f"H{n:06d}"}

    elif n > _S3_MAX_H and n <= _S4_MAX_H:  # Section 4 — K J I L
        entry = _S4_H_LOT.get(n)
        if entry is None: return None
        sec2, lot2 = entry
        return {'section':sec2,'lot_label':lot2,'h_number':n,'h_formatted':f'H{n:06d}'}

    elif n > _S2_MAX_H and n <= _S3_MAX_H:  # Section 3 — M
        entry = _M_H_LOT.get(n)
        if entry is None: return None
        sec2, lot2 = entry
        return {'section':sec2,'lot_label':lot2,'h_number':n,'h_formatted':f'H{n:06d}'}

    elif n > _EF_MAX_H and n <= _S2_MAX_H:  # Section 2 — margins + G/H
        entry = _S2_H_LOT.get(n)
        if entry is None: return None
        sec2, lot2 = entry
        return {'section':sec2,'lot_label':lot2,'h_number':n,'h_formatted':f'H{n:06d}'}

    else:                                   # Pass 3 — F / E (table-driven)
        entry = _EF_H_LOT.get(n)
        if entry is None: return None
        sec2, lot2 = entry
        return {'section':sec2,'lot_label':lot2,'h_number':n,'h_formatted':f'H{n:06d}'}
```

pre `hformula.py · 3 lines · arithmetic rebuild`

The sum term: how many lots all earlier levels of a two-section pass hold.

Implements: Step 3 · rows of different widths

```
def pre(w1, w2, r):
    """lots swept in rows 1..r-1 of a two-section pass whose row widths are w1, w2"""
    return sum((w1[j] if j < len(w1) else 0) + (w2[j] if j < len(w2) else 0) for j in range(r - 1))
```

H_A_grid `hformula.py · 7 lines · arithmetic rebuild`

Section A: label to H using 19 labels per row and three H-lots per row.

Implements: Step 2 · H-lots push labels along

```
def H_A_grid(n, sf):
    if not (1 <= n <= 19 * ROWS): return None
    r = (n - 1) // 19 + 1; q = n - 19 * (r - 1)                                  # increment 19; q = p
 lace inside the row
    if sf == '': c = 20 - q + (q <= 13) + (q <= 5) + (q <= 1)
    elif sf == 'H' and q in (13, 5, 1): c = 20 - q + (q <= 5) + (q <= 1)          # H-lot sits just be
 fore its twin
    else: return None
```

```
    return SW * (r - 1) + N + c, 'closed'
```

H_B_grid hformula.py · 10 lines · arithmetic rebuild

Section B: label to H for the body, the last column and the H-lots.

Implements: Step 2 · Section B rules

```
def H_B_grid(n, sf):
    if sf == 'H' and n % 20 == 0 and 1 <= n // 20 <= ROWS: r, c = n // 20, 1                # H-lots 20H,40H..380H : first column
    elif sf == 'A' and n == 19: r, c = 1, 2                                                 # the one lettered lot "19A"
    elif sf == '':
        rr = next((x for x in range(1, ROWS + 1) if n == 12 + 19 * (x - 1)), None)
        if rr is not None: r, c = rr, N                                                      # last column: 12,31,50,... (increment 19 - wins any clash)
        elif 1 <= n <= 380 and n % 20 != 0: r = (n - 1) // 20 + 1; c = N - (n - 20 * (r - 1))   # body: increment 20, columns 3..21, right-to-left
        else: return None
    else: return None
    return SW * (r - 1) + c, 'closed'
```

H_D hformula.py · 23 lines · arithmetic rebuild

Section D: label to H across uneven rows, blocks and H-lots.

Implements: Step 3 · rows of different widths

```
def H_D(pf, n, sf, lab):
    """Section D, main grid (the margin strip 542-563H is handled in H_margin_D)"""
    base = None
    if lab in ('1U', '1T', '1S', '1R', '1Q', '1P', '1O', '1N', '1M', '1L', '1K', '1J', '1I', '1H'): r, k, tier = 1, 1 + (ord('U') - ord(lab[1])), 'rows'
    elif pf == '' and sf == '' and 1 <= n <= 20: r, k, tier = 1, 15 + (20 - n), 'rows'
    elif lab in ('1A', '1B', '1C', '1D', '1E'): r, k, tier = 1, 35 + (ord(lab[1]) - ord('A')), 'rows'
    elif lab == 'F1': r, k, tier = 1, 40, 'rows'
    elif lab == '1G': r, k, tier = 1, 41, 'rows'
    elif lab == '1H_2': r, k, tier = 1, 42, 'rows'
    elif lab == '1I_2': r, k, tier = 1, 43, 'rows'
    elif pf == '' and sf == 'H' and 20 <= n <= 280 and n % 20 == 0: r, k, tier = n // 20 + 1, 1, 'closed'   # 20H, 40H ... 280H : first lot of rows 2..15
    elif pf == '' and sf == 'H' and 29 <= n <= 289 and n % 20 == 9: r, k, tier = (n + 11) // 20, 13, 'closed'   # 29H, 49H ... 289H : 13th lot
    elif pf == '' and sf == 'H' and any(n == v for v in DHLOT.values()): r = next(x for x, v in DHLOT.items() if v == n); k, tier = 1, 'rows'
    elif pf == '' and sf == '' and 21 <= n <= 300:
        r = (n - 1) // 20 + 1; j = 20 * r - n; k, tier = (j + 2 if j <= 10 else j + 3), 'closed'                # 20 lots per row, 2 H-lots slipped in
    elif pf == '' and sf in ('A', 'B') and 367 <= n <= 384:                                                  # double-row lots: nA and nB share the position of n
        return H_D(pf, n, '', str(n))
    elif pf == '' and sf == '':
        hit = next(((r_, k0 + top - n) for (r_, k0, top, bot) in DBLOCK if bot <= n <= top), None)
        if hit is None: return None
        r, k = hit; tier = 'rows'
    else: return None
    return O_DC + pre(WD, WC, r) + k, tier
```

H_C hformula.py · 12 lines · arithmetic rebuild

Section C: label to H using each row's top lot, twins and trios.

Implements: Step 4 · twins and trios

```
def H_C(pf, n, sf):
    r = next((x for x in range(1, 25) if n <= CT[x - 1]), None)
    if r is None or (r > 1 and n <= CT[r - 2]): return None
    trio = 3 if r in CTRIO else 0
    tw = CTW[r]
    def before(nn): return sum(1 for (t, f) in tw if t > nn or (t == nn and f == 'b'))
    if sf == '': k = (CT[r - 1] - n) + 1 + trio + before(n)
    elif sf in ('C', 'B', 'A') and r in CTRIO and n == CT[r - 1]: k = 'CBA'.index(sf) + 1
    elif sf == 'H' and (n, 'b') in tw: k = (CT[r - 1] - n) + 1 + trio + before(n) - 1
    elif sf == 'H' and (n, 'a') in tw: k = (CT[r - 1] - n) + 1 + trio + before(n) + 1
    else: return None
    return O_DC + pre(WD, WC, r) + (WD[r - 1] if r <= len(WD) else 0) + k, 'rows'
```

H_F hformula.py · 7 lines · arithmetic rebuild

Section F: 18 labels per row, plus the margin lots 199 to 210.

Implements: Step 5 · straight-line sections

```
def H_F(pf, n, sf):
    if sf != '' or pf != '': return None
    if 1 <= n <= 198:
        r = (n - 1) // 18 + 1; c = 19 - (n - 18 * (r - 1))                # increment 18, no H-lots; right-to-le
ft
        return O_EF + pre(WF, WE, r) + c, 'closed'
    if 199 <= n <= 210: return O_MARGIN_F + (n - 198), 'closed'              # margin strip: just 199..210 in a
row
    return None
```

H_E hformula.py · 26 lines · arithmetic rebuild

Section E: 19 per row with twins, plus the lettered lots in rows 9 to 12.

Implements: Step 4 · twins and trios

```
def H_E(pf, n, sf, lab):
    base = lab
    if re.match(r'^(152|169|19\d)[AB]$', lab) and n in (152, 169, 190, 191, 192, 193, 194, 195, 196, 197, 198, 19
9):  # double-row lots share ONE position
        r, k = (9, 21) if n == 152 else ((10, 10) if n == 169 else (10, 12 + (199 - n)))
        return O_EF + pre(WF, WE, r) + WF[r - 1] + k, 'rows'
    if lab == '199BH': return O_EF + pre(WF, WE, 10) + WF[9] + 11, 'rows'
    if pf != '': return None
    if sf == '' and 1 <= n <= 152 and not (n <= 152 and False):
        # rows 1..8 : increment 19, H-lots slipped in before q = 13, 9, 5
        r = (n - 1) // 19 + 1; q = n - 19 * (r - 1)
        k = (19 - q + 1) + sum(1 for t in (13, 9, 5) if t >= q)
        return O_EF + pre(WF, WE, r) + WF[r - 1] + k, 'closed'
    if sf == 'H' and n <= 152 and (n - 19 * ((n - 1) // 19)) in (13, 9, 5):
        r = (n - 1) // 19 + 1; q = n - 19 * (r - 1)
        k = (19 - q + 1) + sum(1 for t in (13, 9, 5) if t >= q) - 1
        return O_EF + pre(WF, WE, r) + WF[r - 1] + k, 'closed'
    if sf == 'H':
        hit = next(((r_, k) for (nn, r_), k in EHLOT.items() if nn == str(n) and r_ >= 9), None)
        if hit: r, k = hit; return O_EF + pre(WF, WE, r) + WF[r - 1] + k, 'rows'
        return None
    if sf == '':
```

```
        hits = [(r_, k0 + top - n) for (r_, k0, top, bot) in EBLOCK if bot <= n <= top]
        if not hits: return None
        r, k = hits[0]                                              # first block that claims the lot, as t
  he decoder does
        return O_EF + pre(WF, WE, r) + WF[r - 1] + k, 'rows'
    return None
```

H_G `hformula.py · 13 lines · arithmetic rebuild`

Section G: 19 per row, plus lots 58 to 75.

Implements: Step 3 · running totals

```
def H_G(pf, n, sf):
    pg = lambda r: sum(WG[j] + WH[j] for j in range(r - 1))
    if sf == '' and n in (58, 59, 60): r = n - 57; k = 1
    elif sf in ('', 'H') and 1 <= n <= 57:
        r = (n - 1) // 19 + 1; top = 19 * r; j = top - n
        if sf == 'H':
            if n == top: k = 2
            elif n == top - 11: k = 14
            else: return None
        else: k = j + 3 if j <= 10 else j + 4
    elif sf == '' and 61 <= n <= 75: r, k = 4, n - 60
    else: return None
    return O_GH + pg(r) + k, 'rows'
```

H_H `hformula.py · 4 lines · arithmetic rebuild`

Section H: 23 per row, four rows.

Implements: Step 3 · running totals

```
def H_H(pf, n, sf):
    if sf != '' or not (1 <= n <= 92): return None
    r = (n - 1) // 23 + 1; k = 23 * r + 1 - n                       # increment 23, right-to-left
    return O_GH + sum(WG[j] + WH[j] for j in range(r - 1)) + WG[r - 1] + k, 'closed'
```

H_M `hformula.py · 5 lines · arithmetic rebuild`

Section M: n + 2556 from lot 82, and a stored list below that.

Implements: Step 5 · straight-line sections

```
def H_M(pf, n, sf):
    if sf != '' or pf != '': return None
    if 82 <= n <= 279: return n + 2556, 'closed'
    if str(n) in MFIRST: return O_M + MFIRST.index(str(n)) + 1, 'list'
    return None
```

H_K `hformula.py · 1 lines · arithmetic rebuild`

Section K: the position of the label in a stored list of 115.

Implements: Step 5 · straight-line sections

```
def H_K(pf, n, sf): return (O_K + KORDER.index(str(n)) + 1, 'list') if sf == '' and pf == '' and str(n) in KORDER
 else None
```

H_J `hformula.py · 1 lines · arithmetic rebuild`

Section J: 2950 + n.

Implements: Step 5 · straight-line sections

```
def H_J(pf, n, sf): return (O_J + n, 'closed') if sf == '' and pf == '' and 1 <= n <= 132 else None
```

H_I `hformula.py · 1 lines · arithmetic rebuild`

Section I: 3082 + n.

Implements: Step 5 · straight-line sections

```
def H_I(pf, n, sf): return (O_I + n, 'closed') if sf == '' and pf == '' and 1 <= n <= 64 else None
```

H_L `hformula.py · 1 lines · arithmetic rebuild`

Section L: 3146 + (n − 68).

Implements: Step 5 · straight-line sections

```
def H_L(pf, n, sf): return (O_L + n - 68, 'closed') if sf == '' and pf == '' and 69 <= n <= 107 else None
```

H `hformula.py · 21 lines · arithmetic rebuild`

The dispatcher: parse the label, call the section's function, apply the known errata.

Implements: Section 07 · the giant formula

```
def H(section, label):
    """(H-number, tier) or (None, why)"""
    S = section.upper(); pf, n, sf = parse(label)
    if n is None: return None, 'unreadable label'
    if (S, label) in ERRATA: return ERRATA[(S, label)]()
    res = None
    if S == 'B': res = H_margin_B(label) or H_B_grid(n, sf) if pf == '' else None
    elif S == 'A': res = H_A_grid(n, sf) if pf == '' else None
    elif S == 'D': res = (H_margin_D(n, sf) if pf == '' else None) or H_D(pf, n, sf, label)
    elif S == 'C': res = H_C(pf, n, sf) if pf == '' else None
    elif S == 'F': res = H_F(pf, n, sf)
    elif S == 'E': res = H_E(pf, n, sf, label)
    elif S == 'G': res = H_G(pf, n, sf)
    elif S == 'H': res = H_H(pf, n, sf)
    elif S == 'M': res = H_M(pf, n, sf)
    elif S == 'K': res = H_K(pf, n, sf)
    elif S == 'J': res = H_J(pf, n, sf)
    elif S == 'I': res = H_I(pf, n, sf)
    elif S == 'L': res = H_L(pf, n, sf)
    if not res and (S, label) in ERRATA: return ERRATA[(S, label)]()
    return res if res else (None, 'no such lot')
```

07

The giant formula

This is the formula the whole system rests on. Every old section letter and lot label goes in, and one H-number comes out. It reproduces the decoder for every real label.

THE H-NUMBER FORMULA

$$H(S,\ell) = O_{p(S)} + \sum_{j<r} (W_j^1 + W_j^2) + \sigma_S \cdot W_r^1 + \kappa_S(\ell)$$

Tap a term to see what it does for the lot below

0

Where the pass starts



The lots in every earlier pass. It only depends on which pass the section belongs to.

APPLIES TO every section

836

Σ

Levels already walked



The lots on all the earlier levels of this pass. Straight runs have no levels, so Σ is 0.

APPLIES TO A to H

194

σ · W

Waiting for the first section



On a level, the second section starts after the first one. σ is 1 for the second section of a pass, 0 for the first.

`APPLIES TO` A, C, E, H

22

K

Place on the level



Turns the old label into a position on its level. Every section has its own recipe, listed below.

`APPLIES TO` every section

7

H(C, 89) = 1059



The bar is the H-number split into its four terms.

3,197
real labels where the formula equals the decoder

3,139
distinct H-numbers they land on, out of 3,185 positions

13
old sections, one formula

6

passes along the path

▶ **How the formula was built, in six steps**

Old address to H-number

SECTION: C

LOT LABEL: 89

H001059 1059 of 3,185

ACCEPTED DECODER LABEL The decoder lists C-89 and gives the same number.

Graves: H0010591 to H0010599.

STEP	OPERATION	VALUE
1	**Parse the label** prefix, number, suffix	**89**
2	**Pick the section rule** Section C: row tops, twins and trios from per-row tables.	**C · row constants**
3	**Find the level and place** decoder indexing, not always the rows on the ground	**level 5 · place 7**
4	**Offset** Lots visited before this level (H minus place): 836 + 194 + 22 (base, earlier levels, this level of D)	**1052**
5	**Add place to offset** H = offset + place	**H001059**

▶ **Show this section's raw expression**

H-number to old label

H-NUMBER

1690

H001690

D-384 D-384A D-384B

3 references share this number. They differ only by an A or B suffix. In the decoder they are the two halves of one double-row lot and its plain label. Graves keep the letter: H001690A1, H001690B1.

κ for each section

κ is the lot's place on its level. Each section turns its old label into a place in its own way.

SECTION	PLACE ON THE LEVEL K(ℓ)	H
A	n → r = ⌈n/19⌉, q = n − 19(r−1); c = 20 − q + [q≤13] + [q≤5] + [q≤1] − [H-lot]	44(r−1) + 22 + c
B	body: r = ⌈n/20⌉, c = 20r − n; last column: r = (n−12)/19 + 1	44(r−1) + 22 + c; H-lot 20r: 44(r−1) + 1
C	k = (top_r − n) + 1 + 3·[trio] + #twins above	836 + Σ(W_D+W_C) + W_D(r) + k
D	by block (r, k_o, hi, lo): k = k_o + hi − n; H-lots by row	836 + Σ(W_D+W_C) + k
E	k = 20 − q + [q≤13] + [q≤9] + [q≤5]; blocks in rows 9 to 12	1871 + Σ(W_F+W_E) + W_F(r) + k
F	k = 19 − (n − 18(r−1))	1871 + Σ(W_F+W_E) + k
G	k = 19r − n + (3 or 4) by side of H-lots	2413 + Σ(W_G+W_H) + k
H	k = 23r + 1 − n	2413 + Σ(W_G+W_H) + W_G(r) + k
M	n + 2556 for n ≥ 82; position in list otherwise	2586 + position
K	position of n in an ordered list of 115	2835 + position
J	n	2950 + n
I	n	3082 + n
L	n − 68	3146 + n − 68

The one-line version

This is the exact text the calculator runs. Paste it into a JavaScript console and call `H("B","20H")`.

giant_formula.js constants first, then one return expression

```
H = (S, L) => {
  var m,p,n,f,r,q,c,k,i,j,g,b;
  BM=["373","374","375","376","376H","377","378","379","380","380H","381","382","383","384","384HA","384H","385A","38
5","386A","386","387B","387A","387","388B","388A","388","388HB","388HA","388H","389B","389A","389","390C","390B","390
A","390","390H","391","392","393","394"];
  CT=[19,38,57,76,95,114,133,152,171,190,209,228,247,263,279,295,311,330,349,368,384,400,416,432];
  TW=[[[9,0],[5,0]],[[32,0],[24,0]],[[51,0],[47,0],[43,0]],[[66,0],[62,0]],[[85,0],[81,0]],[[108,0],[104,0],[100,0]],
[[127,0],[123,0],[119,0]],[[146,0],[142,0],[138,0]],[[165,0]],[[184,0],[180,0],[176,0]],[[203,0],[199,0],[195,0]],[[22
2,0]],[[241,1],[233,0]],[[260,0],[256,0],[252,0]],[[276,1],[272,0],[268,0]],[[292,0],[288,0],[284,0]],[[308,0],[304,
0],[300,0]],[[324,0],[320,0],[316,1]],[[343,1],[339,0],[335,1]],[[362,0],[358,0],[354,1]],[[381,0],[377,0],[373,1]],
[[397,1],[393,0],[389,1]],[[413,0],[409,0],[405,1]],[[429,1],[425,0],[421,1]]];
  TR=[0,0,0,0,0,0,0,0,0,0,0,0,1,1,1,1,0,0,0,1,1,1,1];
  P2=[0,64,107,151,194,237,281,325,369,411,455,499,541,584,628,672,714,756,798,841,884,927,970,1013];
  WD=[43,22,22,22,22,22,22,22,22,22,22,22,22,22,22,22,20,20,20,21,21,21,21,21,0];
  P3=[0,40,80,120,160,200,240,280,320,359,398,438];
  WF=[18,18,18,18,18,18,18,18,18,18,18,18,0];
  PG=[0,45,90,135];
  WG=[22,22,22,15];
  DM=[545,549,556,559,560,562,563];
  DH=[311,322,333,344,355,366];
  DB=[[16,2,310,301],[16,12,420,412],[17,2,321,312],[17,12,411,403],[18,2,332,323],[18,12,402,394],[19,2,344,344],[19,
3,343,334],[19,13,393,385],[20,2,355,355],[20,3,354,345],[20,13,384,376],[21,2,366,366],[21,3,365,356],[21,13,375,36
7],[22,1,541,521],[23,1,520,500]];
  EB=[[9,1,169,164],[9,8,163,163],[9,9,162,160],[9,13,159,159],[9,14,158,156],[9,18,155,155],[9,19,154,153],[10,1,177,
172],[10,8,171,171],[10,9,170,170],[10,12,199,190],[11,1,183,178],[11,8,213,210],[11,13,209,209],[11,14,208,200],[12,
1,189,184],[12,8,227,224],[12,13,223,223],[12,14,222,214]];
  EH=[[163,9,7],[159,9,12],[155,9,17],[171,10,7],[177,11,7],[209,11,12],[183,12,7],[223,12,12]];
  MF=["11","12","14","15","16","18","19","20","23","24","25","28","29","30","31","34","35","36","37","40","41","42","4
3","46","47","48","49","52","53","54","55","61","58","59","60","66","67","64","65","71","72","73","74","75","70","7
6","77","78","79","80","81"];
  KO=["1","2","3","6","7","8","9","13","14","15","16","22","5","11","12","18","19","25","26","31","32","36","37","3
8","42","43","44","45","50","51","52","57","58","59","64","65","66","72","73","79","80","86","87","93","94","101","10
2","109","110","117","118","125","126","133","134","141","142","149","150","158","159","167","168","176","177","18
5","186","195","196","205","206","215","216","225","226","235","236","246","247","256","257","258","259","268","26
9","270","271","280","281","282","283","293","294","295","296","306","307","308","309","319","320","321","322","33
2","333","334","335","345","346","347","348","358","359","360","361"];
  return (m=/^([A-Z]?)([0-9]+)(.*)$/.exec(L),p=m?m[1]:'',n=m?+m[2]:NaN,f=m?m[3]:'',S==='A'?(r=Math.floor((n-1)/19)+1,q
=n-19*(r-1),(p===''&&n>=1&&n<=361&&(f===''||(f==='H'&&(q===13||q===5||q===1))))?44*(r-1)+22+(20-q+(q<=13)+(q<=5)+(q<=
1)-(f==='H')):NaN):S==='B'?((i=BM.indexOf(L))>=0?2331+i+1:p!==''?NaN:f==='H'?((n%20===0&&n>=20&&n<=380)?44*(n/20-1)+1:
NaN):L==='19A'?2:f!==''?NaN:((r=(n-12)/19+1,Number.isInteger(r)&&r>=1&&r<=19)?44*(r-1)+22:((n>=1&&n<=380&&n%20!==0)?44
*Math.floor((n-1)/20)+22-(n-20*Math.floor((n-1)/20)):NaN))):S==='C'?(L==='124H'&&(n=142),r=CT.findIndex(t=>n<=t)+1,(p=
=''&&r>0&&(r===1||n>CT[r-2]))?(b=TW[r-1].filter(w=>w[0]>n||(w[0]===n&&!w[1])).length,k=CT[r-1]-n+1+3*TR[r-1]+b,f==
=''?836+P2[r-1]+WD[r-1]+k:(f==='H'&&TW[r-1].some(w=>w[0]===n))?836+P2[r-1]+WD[r-1]+k+(TW[r-1].find(w=>w[0]===n)[1]?1:-
1):(f.length===1&&'CBA'.indexOf(f)>=0&&TR[r-1]&&n===CT[r-1])?836+P2[r-1]+WD[r-1]+'CBA'.indexOf(f)+1:NaN):NaN):S==='D'?
((f==='A'||f==='B')&&n>=367&&n<=384&&(f=''),(p===''&&n>=542&&n<=563&&(f===''||(f==='H'&&DM.includes(n))))?2372+(n-541)
+DM.filter(t=>t<n).length+(f==='H'?1:0):(/^1[H-U]$/.test(L))?836+1+(85-L.charCodeAt(1)):(p===''&&f===''&&n>=1&&n<=20)?
836+15+(20-n):(L==='1A'||L==='1B'||L==='1C'||L==='1D'||L==='1E')?836+35+(L.charCodeAt(1)-65):L==='F1'?836+40:L==='1G'?
836+41:L==='1H_2'?836+42:L==='1I_2'?836+43:(p===''&&f==='H')?((n%20===0&&n>=20&&n<=280)?836+P2[n/20]+1:(n%20===9&&n>=2
9&&n<=289)?836+P2[(n+11)/20-1]+13:DH.includes(n)?836+P2[DH.indexOf(n)+15]+1:NaN):(p===''&&f===''&&n>=21&&n<=300)?(r=Ma
th.floor((n-1)/20)+1,j=20*r-n,836+P2[r-1]+(j<=10?j+2:j+3)):(p===''&&f===''&&(g=DB.find(x=>x[3]<=n&&n<=x[2]))?836+P2[g
[0]-1]+g[1]+g[2]-n:NaN):S==='E'?(/^(152|169|19[0-9])[AB]$/.test(L)&&(n===152||n===169||(n>=190&&n<=199))?(r=n===152?9:
10,k=n===152?21:n===169?10:12+(199-n),1871+P3[r-1]+WF[r-1]+k):L==='199BH'?1871+P3[9]+WF[9]+11:p!==''?NaN:(f===''&&n>=1
```

```
&&n<=152)?(r=Math.floor((n-1)/19)+1,q=n-19*(r-1),1871+P3[r-1]+WF[r-1]+(19-q+1)+(q<=13)+(q<=9)+(q<=5)):(f==='H'&&n<=152
&&((q=n-19*Math.floor((n-1)/19))===13||q===9||q===5))?(r=Math.floor((n-1)/19)+1,1871+P3[r-1]+WF[r-1]+(19-q+1)+(q<=13)+
(q<=9)+(q<=5)-1):f==='H'?((g=EH.find(x=>x[0]===n))?1871+P3[g[1]-1]+WF[g[1]-1]+g[2]:NaN):(f===''&&(g=EB.find(x=>x[3]<=n
&&n<=x[2])))?1871+P3[g[0]-1]+WF[g[0]-1]+g[1]+g[2]-n:NaN):S==='F'?((p===''&&f===''&&n>=1&&n<=198)?(r=Math.floor((n-1)/1
8)+1,1871+P3[r-1]+19-(n-18*(r-1))):(p===''&&f===''&&n>=199&&n<=210)?2401+(n-198):NaN):S==='G'?(L==='19'?2415:p!==''?Na
N:(f===''&&(n===58||n===59||n===60))?2413+PG[n-58]+1:((f===''||f==='H')&&n>=1&&n<=57)?(r=Math.floor((n-1)/19)+1,j=19*r
-n,f==='H'?(n===19*r?2413+PG[r-1]+2:n===19*r-11?2413+PG[r-1]+14:NaN):2413+PG[r-1]+(j<=10?j+3:j+4)):(f===''&&n>=61&&n<=
75)?2413+PG[3]+(n-60):NaN):S==='H'?((p===''&&f===''&&n>=1&&n<=92)?(r=Math.floor((n-1)/23)+1,2413+PG[r-1]+22*0+WG[r-1]+
23*r+1-n):NaN):S==='M'?((p===''&&f===''')?((n>=82&&n<=279)?n+2556:(i=MF.indexOf(String(n)))>=0?2586+i+1:NaN):NaN):S==
='K'?((p===''&&f===''&&(i=KO.indexOf(String(n)))>=0)?2835+i+1:NaN):S==='J'?((p===''&&f===''&&n>=1&&n<=132)?2950+n:Na
N):S==='I'?((p===''&&f===''&&n>=1&&n<=64)?3082+n:NaN):S==='L'?((p===''&&f===''&&n>=69&&n<=107)?3146+n-68:NaN):NaN);
}
```

Change the constants and re-check EXPERIMENTAL CONSTANTS

These are the formula's constants: row widths, twin lists, blocks and base offsets. Edit them and press Apply. The page re-runs every label and reports how many H-numbers move. The edits affect this demonstration only.

```
All 3,197 real labels still match the decoder.
```

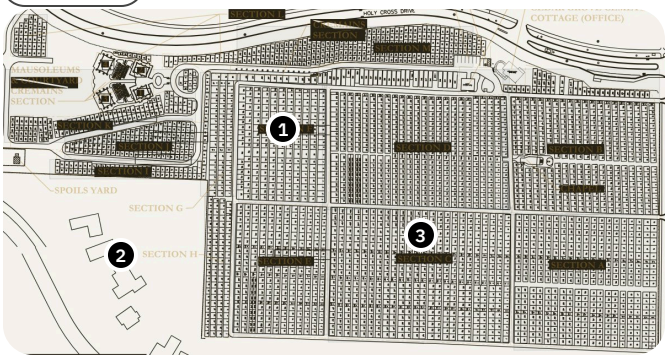
08

Old and new, side by side

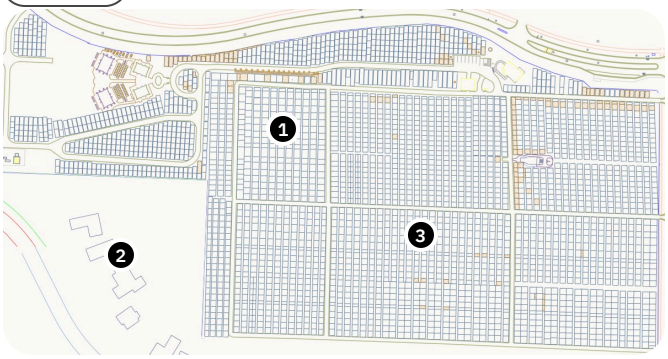
Each pair uses the same crop, scale and orientation. Drag the handle to compare, zoom to look closer (both images zoom together), or switch to side by side.

Whole map

BEFORE



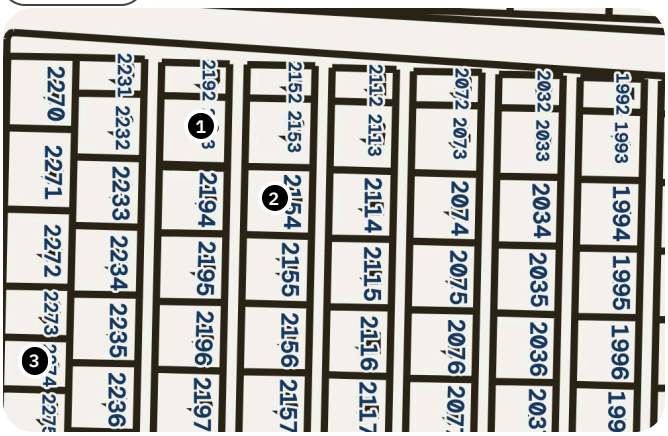
AFTER



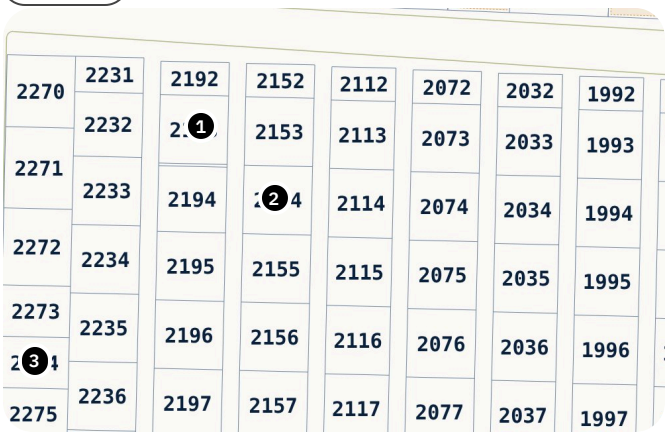
1. Before: dark title blocks sit on top of the lots they name, and the legacy notes crowd the margins.
2. After: no title blocks over lots. Roads, buildings and lot outlines are kept in muted colors.
3. Outlines are thin, so the lot grid is readable at this scale.

Straight rows

BEFORE



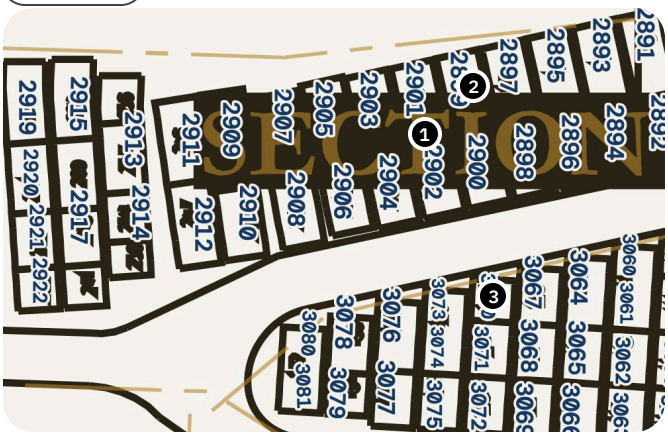
AFTER



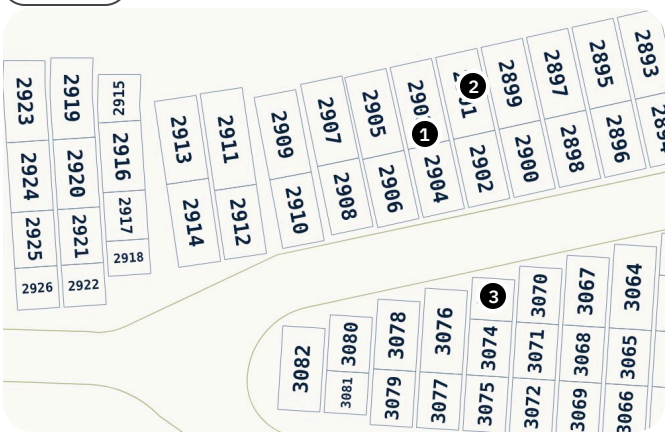
1. Before: each number is drawn as an outlined shape, and the old and new numbers overlap (look at 2193 and 2033).
2. After: one number per lot, in a clean typeface, inside the lot outline.
3. Contrast improved: dark numerals on a plain lot face. Small lots get smaller numbers.

Curved rows and a section title

BEFORE



AFTER



1. Before: the section title ("SECTION") runs across the numbers, and numbers cross lot edges.
2. After: the title is gone and each number sits inside its lot.
3. In the straight blocks the numbers read left to right. In curved rows they turn with the lot, so the reading direction follows the row.

What changed, in short

QUESTION	LEGACY LABELS	H-NUMBERS
One name per lot	A section plus a label names a lot. The label alone does not: 474 of 768 label strings are used by two or more sections.	One number from 1 to 3,185 names a position. Graves add one digit, and a double-row half adds A or B.
Same rule everywhere	No. Each of the 13 sections counts in its own way (section 03).	One rule: position along the path.
Order	Restarts in every section and jumps within one.	Rises by 1 along the path, by construction.
Reading it on the map	Tiny, sideways, sometimes carrying another section's letter. The overall map has no lot numbers.	Larger, inside the lot, with the section titles removed.
Finding a lot from the number	Needs the section, then that section's rule.	A single numeric lookup. Expected to be faster; not measured.
Fewer mistakes	Several known ambiguities (section 03, C).	Expected to help, since there is one format. Not measured.
Where the complexity lives	In the labels people read.	In the conversion tables that map old labels to H-numbers (section 07). Those tables still hold the exceptions.

Faster lookup and fewer mistakes are expected benefits of this change. They have not been measured.

09

Checks and errata

Three different questions need three different kinds of evidence. Only the first has been done.

Verification ledger

CHECK	RESULT	EVIDENCE	UNRESOLVED
Software agreement Python formulas against the decoder	3,199 / 3,199	verify.py, stored result	None. Includes the two internal keys.
Software agreement one-line JavaScript against the decoder	3,197 / 3,197	test_giant.js, stored result	Two internal keys left out.
Software agreement live in this page	3,197 / 3,197	Run in your browser when the page loaded	None.
Mapping completeness	3,139 of 3,185	3,197 real labels map to 3,139 distinct H-numbers	46 positions have no legacy reference.
Physical verification	Not done	None yet	A field check of a sample from every pass.

Software agreement, in detail

Coverage by kind across the 3,199 entries: 2039 arithmetic, 951 row constants, 207 stored lists, 2 errata. The two internal keys are `152dr` and `169dr` in Section E. They are bookkeeping entries, not lots.

Mapping completeness

3,197 real labels map to **3,139** distinct H-numbers, because 30 H-numbers have more than one reference: 28 double-row lots with three references each (n, nA, nB) and 2 with two. Of 3,185 positions, **46** have no legacy reference.

36 are unlabeled grid positions in the B/A sweep found by the analysis.

10 46 55 90 100 134 145 178 190 222 235 266 280 310 325 354 370 398

10 more, so the 36 are only part of the total.

793 795 796 797 798 799 800 801 1473 2416

H000793 and H000795 to H000801 sit at the end of pass 1. H001473 is in C and H002416 is in

415	442	460	486	505	530
531	574	576	618	621	662
666	706	711	750	756	794

the G margin.

Errata

ISSUE	AFFECTED REFERENCES	CONSEQUENCE	CURRENT HANDLING	DECISION NEEDED
Duplicate keys	D-274, G-19	One of each pair is shadowed	Decoder keeps the later entry; formula follows it	Decide which lot each entry meant
Typo in a row table	C-124H (pattern says 142H)	A lookup for C-142H finds nothing	Formula maps 124H to the 142H position	Confirm on the map
Labels claimed twice	B-373 to B-379	Grid positions lose their own label	Later rule (the margin list) wins	Rule which lot each label means
Internal keys	E-152dr, E-169dr	Counted in 3,199, not real lots	Left out of the live check	Remove from the decoder or document them
Traversal order	B and A, pass 1	Document says alternate columns; code does whole levels	Code order used; every verified number follows it	Choose the authoritative order
Idealised grid	C, reference sheet	C-417 is H001672 on the sheet and H001871 in the decoder	Decoder uses the real rows	Retire or update the sheet
Pass boundary	H and M	Earlier summaries put M at H002586; it begins at H002587	This page computes pass ranges from the data	Check other documents that quote it

”Later entry wins” describes what the software does. It does not show that the underlying conflict is settled.

Decisions needed

#	DECISION	OPTIONS AND CONTEXT	IF LEFT OPEN
1	**Which traversal order is authoritative?**	Whole level of B then A (what the code does) or alternating columns (what the reference PDF says).	Everything in pass 1 depends on it. Until it is decided, H-numbers in B and A are provisional.
2	**Which exceptions get reviewed first?**	Suggested order: the 5 label conflicts in the errata table, then the 46 positions with no reference, then the double-row lots.	Unreviewed conflicts keep some lots unreachable by their old label.
3	**How will the numbers be verified on the ground?**	Someone has to walk a sample of lots in each pass and confirm that the H-number sits on the right lot.	Without it, the numbers are checked against the decoder only.
4	**What are the rules for future additions?**	How a new lot or a re-cut lot gets a number without disturbing the order: append after H003185, insert and renumber, or reserve gaps.	Without a rule, the first addition forces an ad hoc choice.

Reproduce it

```
# from the repository root
python3 h-number-map/math/extract_decoder.py   # dump every H-number and label the decoder knows
python3 h-number-map/math/verify.py            # compare hformula.py with the decoder: 3,199 of 3,199
python3 h-number-map/math/gen_giant.py         # compile the one-line expression
node h-number-map/math/test_giant.js           # check the one-line expression: 3,197 of 3,197 real labels
```